

Project 3: Postgres CDC Pipeline with CI/CD

Change data capture into a lakehouse, with the engineering practices around it

Streaming change data capture from Postgres into a Databricks lakehouse, built with the practices that make a pipeline maintainable: testable transformation functions, unit tests, linting, and a CI/CD workflow that deploys it.

This document contains

- Part D: the full Project 3 build (manual pages 40-51)
- Part A shared set-up and naming standards (manual pages 3-12)
- Parts F and G: finishing, publishing, troubleshooting and the full command reference (manual pages 59-64)

Vivek Pandey — Data Engineer

Azure · Databricks · PySpark · Delta Lake

vp171097.github.io/DevWebsite

PART D

Project 3: Postgres CDC Pipeline with CI/CD

Client story: reporting queries hit the production PostgreSQL database and slow the app. The client wants changes synced to Databricks with history, deployed through CI/CD with dev and prod.

Duration: about 12 days.

D1. Repository layout

```
databricks-postgres-cdc-cicd/
├── README.md
├── databricks.yml
├── resources/  cdc_pipeline.yml  cdc_job.yml
├── source_db/  schema.sql  README.md
├── simulator/  seed_source.py  generate_changes.py
├── extractor/  extract_changes.py  state/.gitkeep
├── src/cdc/
│   ├── __init__.py  config.py
│   ├── transformations/  __init__.py  changes.py  customers.py  orders.py
│   ├── pipelines/  bronze.py  silver.py  gold.py
│   ├── notebooks/  freshness_check  reconcile_with_source
│   ├── tests/  conftest.py  test_changes.py  test_customers.py
│   ├── .github/
│   │   ├── workflows/  ci.yml  deploy.yml
│   │   └── pull_request_template.md
│   └── docs/  architecture.png  screenshots/
└── .env.example  .gitignore  pyproject.toml  requirements-dev.txt  LICENSE
```

► Terminal · repo root

```
cd ~/portfolio
git clone git@github.com:[your-github-username]/databricks-postgres-cdc-cicd.git
cd databricks-postgres-cdc-cicd
mkdir -p resources source_db simulator extractor/state src/cdc/transformations \
    src/cdc/pipelines \
    notebooks tests .github/workflows docs/screenshots
touch extractor/state/.gitkeep
# requirements-dev.txt extras for this project: faker, psycopg[binary]
python3.11 -m venv .venv && source .venv/bin/activate && pip install -r requirements-dev.txt
```

.env.example keys for this project: PG_CONNECTION, DATABRICKS_CONFIG_PROFILE, CDC_ENV (dev or prod), LANDING_BASE (/Volumes/portfolio/cdc_landing/changes).

D2. Day 1: source database

Option A: Neon (hosted, recommended)

- 1 Browser: neon.tech → sign up → New project → name shopdb-demo, Postgres 16, nearest region.
- 2 Create database shopdb. Copy the connection string (it includes sslmode=require) into .env as PG_CONNECTION.

Option B: Docker (local)

► Terminal · repo root

```
docker run --name shopdb -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=shopdb \
    -p 5432:5432 -d postgres:16
# PG_CONNECTION=postgres://postgres:postgres@localhost:5432/shopdb
docker ps
# container 'shopdb' is Up
```

Schema (source_db/schema.sql)

Table	Columns and constraints
customers	customer_id bigint identity primary key · email text not null unique · full_name text · city text · country text · segment text check in (retail, business, vip) · is_deleted boolean default false · created_at timestamptz default now() · updated_at timestamptz default now()
orders	order_id bigint identity primary key · customer_id bigint references customers · status text check in (placed, paid, shipped, delivered, cancelled) · amount numeric(10,2) · currency char(3) default EUR · is_deleted boolean default false · created_at · updated_at
Trigger	A trigger function that sets updated_at = now() before every update, attached to both tables
Indexes	On updated_at for both tables (the extractor filters on it); on orders.customer_id

► Terminal · repo root

```
set -a && source .env && set +a                                # load PG_CONNECTION into the shell
psql "$PG_CONNECTION" -f source_db/schema.sql
psql "$PG_CONNECTION" -c "\dt"                                # lists customers, orders
psql "$PG_CONNECTION" -c "\d+ customers"                      # columns, trigger, indexes
```

D3. Day 2: simulators

simulator/seed_source.py

Arguments	--customers 5000 --orders 20000 --seed 7 --reset
Logic	1. Load .env; connect with psycopg using PG_CONNECTION. 2. If tables already contain rows and --reset isn't given, stop. With --reset, truncate orders then customers and restart their identity sequences. 3. Seed Faker and Python's random with the seed. 4. Generate customers: unique email (Faker unique), full name, city and country from a list of 10 European countries, segment with weights 70/25/5. 5. Insert customers in batches of 1,000 (executemany inside a transaction); fetch the generated ids. 6. Generate orders for random customer ids: amount log-normal between 5 and 2,000 rounded to 2 decimals; status weighted toward delivered; created_at random within the last 90 days; updated_at = created_at. 7. Insert orders in batches of 1,000. 8. Print counts per table and per status.

simulator/generate_changes.py

Arguments	--duration-min 10 --rate 5 (actions per second) --mix 50,40,10 (insert/update/delete %) --seed 11
Logic	1. Connect; load the current ids of non-deleted customers and orders into memory. 2. Loop until the duration ends: pick an action by the mix; sleep 1/rate seconds between actions. 3. Insert (split 30% customer / 70% order): create a new customer or a new placed order for a random existing customer; add the new id to the in-memory list. 4. Update (split 30% customer / 70% order): customer → change city or segment; order → move status one step forward (placed→paid→shipped→delivered; 5% of placed or paid become cancelled). Skip orders already final. 5. Delete: set is_deleted = true on a random customer (and all their orders) or on a random order; remove it from memory. 6. Commit after every action (each change gets its own updated_at). 7. Every 100 actions print running counts; at the end print totals per action type.

► Terminal · repo root

```
python simulator/seed_source.py --customers 5000 --orders 20000 --seed 7
psql "$PG_CONNECTION" -c "select count(*) from customers; select status, count(*) from orders \
group by 1;"
python simulator/generate_changes.py --duration-min 2 --rate 5 --mix 50,40,10
psql "$PG_CONNECTION" -c "select count(*) filter (where is_deleted) as deleted, \
max(updated_at) from orders;"
git add -A && git commit -m "feat: source schema and simulators" && git push
```

D4. Days 3-4: change extractor

extractor/extract_changes.py

Arguments	--tables customers,orders --env dev prod --full (ignore the watermark) --dry-run
Outputs	[LANDING_BASE]/[env]/[table]/[table]_[YYYYMMDDTHHMMSSffffff].jsonl and [LANDING_BASE]/[env]/_stats/source_stats_[timestamp].json, where LANDING_BASE = /Volumes/portfolio/cdc_landing/changes
Logic	1. Load .env; connect to Postgres; create the Databricks SDK client from the profile. 2. Read extractor/state/watermark_[env].json → {table: {"updated_at": ISO timestamp, "id": last id}}. Missing or --full → start from 1970-01-01, id 0. 3. Query the database time once: upper_bound = now() minus 2 seconds (a safety margin for transactions still committing). 4. For each table: select rows where (updated_at, primary key) is greater than (watermark updated_at, watermark id) and updated_at is at most upper_bound, ordered by updated_at then primary key. Fetch in chunks of 5,000 with a server-side cursor. 5. For each row add op = D if is_deleted else U; seq = updated_at as microseconds since epoch (a long; it only has to increase per key, and the trigger gives each change of a row a later updated_at); extracted_at = now (UTC). Serialize timestamps as ISO-8601 UTC strings and numerics as strings (no floating-point drift). 6. Write the rows to a local temp .jsonl file (one JSON object per line). If there are none, skip the table. 7. Upload the file to the Volume path. Only after a successful upload, update the in-memory watermark to the last row's (updated_at, id). 8. After all tables: write source_stats JSON with, per table, the count of non-deleted rows, sum of amount for orders, and upper_bound; upload it to _stats/. 9. Save the watermark file atomically (write a temp file, then rename). 10. Log per table: rows, file, new watermark. Exit non-zero on any failure (the watermark is not saved on failure, so the next run re-sends: at-least-once).
Done when	The first run sends all rows; a second run with no activity sends nothing; after the simulator runs, only changed rows are sent.

► Terminal · repo root

```
python extractor/extract_changes.py --env dev --full           # initial load
python extractor/extract_changes.py --env dev                 # expect: 0 rows
python simulator/generate_changes.py --duration-min 1
python extractor/extract_changes.py --env dev                 # only the changes
cat extractor/state/watermark_dev.json
databricks fs ls dbfs:/Volumes/portfolio/cdc_landing/changes/dev/customers/
```

DESIGN NOTE FOR THE README

Watermark extraction can't see **hard deletes**, hence the soft-delete column. For real clients you'd recommend log-based CDC (Debezium, or Databricks Lakeflow Connect for supported databases). Explaining this trade-off shows senior judgment.

TIP

Optional automation: a scheduled GitHub Actions workflow can run the extractor every 15 minutes, using PG_CONNECTION and Databricks credentials stored as secrets. The watermark then needs to live in the Volume instead of the runner's disk: download it at the start, upload it at the end.

D5. Days 5-6: the pipeline

Table ([p] = cdc for prod, cdc_dev for dev)	Type	Logic
[p]_bronze.customer_changes	Streaming table	Auto Loader, format json, explicit schema (no inference), rescue on; + _ingested_at, _source_file
[p]_bronze.order_changes	Streaming table	Same for orders
[p]_silver.customers	AUTO CDC · SCD1	keys customer_id; sequence by seq; apply as delete when op = D; exclude op, seq, extracted_at, technical columns
[p]_silver.customers_history	AUTO CDC · SCD2	Same source and keys; track history on city, segment, email
[p]_silver.orders	AUTO CDC · SCD1	keys order_id
[p]_silver.orders_history	AUTO CDC · SCD2	track history on status, amount
[p]_gold.customer_order_summary	Materialized view	customers left join orders: order_count, delivered_count, total_amount, last_order_at, is_vip (added in the demo PR)
[p]_gold.order_status_durations	Materialized view	From orders_history: per order and status, hours = (__END_AT or now) - __START_AT
[p]_gold.daily_orders	Materialized view	Per created date: new, delivered, cancelled, revenue (delivered amount)

src/cdc/transformations/changes.py

Functions	- normalize_changes(df): upper-case and trim op; cast seq to long; parse timestamps; cast amount to decimal(10,2). - split_valid(df): returns (valid, rejected); valid = op in (U, D) and key and seq not null. - dedupe_exact(df, keys): drop rows with identical (key, seq) (the at-least-once re-sends).
------------------	---

src/cdc/pipelines/silver.py

Logic	1. Read the schema prefix from pipeline configuration key cdc.schema_prefix (cdc or cdc_dev). 2. Declare a temporary view per entity: stream from bronze → normalize_changes → expectations: valid_op (op in U, D) and valid_key (key and seq not null), action drop → dedupe_exact. 3. Declare the four target streaming tables and two AUTO CDC flows per entity (SCD1 and SCD2) from the view.
--------------	---

► SQL editor · verify

```
-- after a pipeline run (dev): compare with Postgres
SELECT COUNT(*) FROM portfolio.cdc_dev_silver.customers;           -- active customers
SELECT customer_id, city, segment, __START_AT, __END_AT
FROM portfolio.cdc_dev_silver.customers_history
WHERE customer_id IN (SELECT customer_id FROM portfolio.cdc_dev_silver.customers_history
                     GROUP BY 1 HAVING COUNT(*) >= 3 LIMIT 1)
ORDER BY __START_AT;
```

► Terminal · repo root

```
# must equal the SQL count above
psql "$PG_CONNECTION" -c "select count(*) from customers where not is_deleted;"
```

D6. Day 7: unit tests

Test	Arrange → expected
normalize op	" u ", "d" → U, D
reject invalid	op X, missing key, missing seq → all in rejected; valid rows untouched
dedupe_exact	same key and seq twice → one row; same key, different seq → two rows
amount precision	"19.99" → 19.99 decimal (not 19.989999...)
customer cleaning	email " A@B.com " → "a@b.com"; unknown segment rejected

► Terminal · repo root

```
pip install pytest-cov
pytest -q --cov=src/cdc/transformations --cov-report=term-missing --cov-fail-under=70
git add -A && git commit -m "test: transformation unit tests" && git push
```

D7. Day 8: Asset Bundle with dev and prod

Setting	dev	prod
mode	development	production
default	true	false
variable schema_prefix	cdc_dev	cdc
variable landing_path	/Volumes/portfolio/cdc_landing/changes/dev	/Volumes/portfolio/cdc_landing/changes/prod
workspace.root_path	automatic (your user folder)	/Workspace/Shared/.bundle/postgres_cdc/prod
job schedule	paused automatically in development mode	every 30 minutes (quartz: 0 0/30 * * * ?)
run_as	you	a service principal if allowed; otherwise you (document it)

► SQL editor

```
-- dev schemas
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_dev_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_dev_silver;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_dev_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_dev_ops;
```

Write databricks.yml (bundle name postgres_cdc, include resources/*.yml, variables schema_prefix, landing_path, alert_email, the two targets), resources/cdc_pipeline.yml (pipeline cdc_pipeline: serverless, catalog portfolio, libraries = the three pipeline files, configuration cdc.schema_prefix and cdc.landing_path from variables) and resources/cdc_job.yml (job cdc_job: tasks run_pipeline → freshness_check and reconcile, schedule, failure email).

► Terminal · repo root

```
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev cdc_job
databricks bundle validate -t prod
# once, to prove it works; afterwards only CI deploys prod
databricks bundle deploy
-t prod
databricks bundle summary -t prod
git add -A && git commit -m "feat: asset bundle with dev and prod targets" && git push
```

D8. Days 9-10: GitHub Actions

Secrets and settings (browser or gh CLI)

► Terminal · repo root

```
gh auth login
gh secret set DATABRICKS_HOST --body "https://[your-workspace-url]"
# paste a token when prompted (fallback auth)
gh secret set DATABRICKS_TOKEN
# preferred, if service principals are available:
gh secret set DATABRICKS_CLIENT_ID --env production
gh secret set DATABRICKS_CLIENT_SECRET --env production
gh secret list
```

- 1 Browser: repo → **Settings** → **Environments** → **New environment** production → Required reviewers: yourself → Deployment branches: main only.
- 2 Browser: **Settings** → **Branches** → **Add branch protection rule** for main: require a pull request, require status checks (select the CI job after its first run), block force pushes.
- 3 Create .github/pull_request_template.md with headings: What changed · Why · How tested · Screenshots · Checklist.

ci.yml: logic of each step

Step	Detail
Trigger	pull_request to main; also workflow_dispatch for manual runs
Checkout	actions/checkout
Python 3.11	actions/setup-python with pip cache
Java 17	actions/setup-java, distribution temurin (needed by local PySpark)
Install	pip install -r requirements-dev.txt
Lint	ruff check . and ruff format --check .
Tests	pytest with coverage, fail under 70% on src/cdc/transformations
Databricks CLI	databricks/setup-cli action
Validate	databricks bundle validate -t dev, with DATABRICKS_HOST and DATABRICKS_TOKEN from secrets as environment variables

deploy.yml: logic

Job	Detail
deploy_dev	On pull_request (after CI succeeds via needs, or by including the test steps): setup CLI → bundle deploy -t dev → bundle run -t dev cdc_job → write the run URL to the job summary
deploy_prod	On push to main: environment production (approval gate) → setup CLI → bundle validate -t prod → bundle deploy -t prod

► Terminal · repo root

```
git checkout -b feature/ci-cd
git add -A && git commit -m "ci: lint, tests, bundle validate and deployments" && git push -u \
  origin feature/ci-cd
gh pr create --fill                                # or open the PR in the browser
gh pr checks --watch                               # watch CI
gh run list --limit 5
```

D9. Day 11: monitoring notebooks

notebooks/freshness_check

Parameters	schema_prefix, max_lag_minutes (default 60)
Logic	1. Read max(_ingested_at) from both bronze tables and max(seq) from both SCD1 tables. 2. lag_minutes = now – max(_ingested_at) for each table. 3. Append (table, last_ingested_at, lag_minutes, checked_at, passed) to [prefix]_ops.freshness_log. 4. Raise an exception if any lag exceeds max_lag_minutes (the job fails and sends the email).

notebooks/reconcile_with_source

Parameters	schema_prefix, landing_path, tolerance (default 0)
Logic	1. Read the newest source_stats file from landing_path/_stats/. 2. Compute in Databricks: active customers (silver.customers row count), active orders, sum of amount. 3. Allow for changes after the snapshot: compare only if the latest applied seq is at or after the snapshot's upper_bound; otherwise record status "skipped: target behind snapshot". 4. Append the comparison to [prefix]_ops.reconciliation_log; raise if any difference exceeds tolerance.

► SQL editor

```
SELECT * FROM portfolio.cdc_dev_ops.freshness_log ORDER BY checked_at DESC LIMIT 10;
SELECT * FROM portfolio.cdc_dev_ops.reconciliation_log ORDER BY checked_at DESC LIMIT 10;
```

D10. Day 12: end-to-end demo

► Terminal · repo root

```
python simulator/seed_source.py --reset --customers 5000 --orders 20000
python extractor/extract_changes.py --env dev --full
databricks bundle run -t dev cdc_job
python simulator/generate_changes.py --duration-min 5
python extractor/extract_changes.py --env dev
databricks bundle run -t dev cdc_job                # screenshot history and status durations

git checkout main && git pull && git checkout -b feature/add-vip-flag
# add is_vip (total_amount >= 1000 or segment = 'vip') to customer_order_summary + a unit test
pytest -q && git add -A && git commit -m "feat: add VIP flag to customer summary"
git push -u origin feature/add-vip-flag && gh pr create --fill
gh pr checks --watch                                # screenshot CI + dev deploy
# then approve 'production' in the Actions tab; screenshot
gh pr merge --squash --delete-branch
```

Finally, stop extracting for over an hour and run the prod job: the freshness check fails and emails you. Screenshot the alert. Add the CI badge to the README.

D11. Azure track: Azure SQL source, Function App extractor, ADLS, Cosmos DB serving

This is the version Azure clients recognise: the application database is **Azure SQL Database** with **Change Tracking**, the extractor runs as a timer **Function App** with a managed identity, change files land in **ADLS Gen2**, secrets live in **Key Vault**, and the gold customer summary is served from **Cosmos DB**. CI/CD logs in to Azure with OpenID Connect (no stored passwords). Prerequisite: A9 done.

Component	Main version	Azure track
Source DB	Neon PostgreSQL	Azure SQL Database shopdb (free offer) with Change Tracking
Deletes	Soft delete column required	Real deletes captured by Change Tracking (operation D)
Extractor	Python script on laptop, updated_at watermark	Function App timer (every 5 min), change-tracking version watermark
Landing	Managed Volume	External Volume on abfss://landing@[ST]/cdc (same Volume name)
Serving	Gold views	Gold + Cosmos DB container customer_360 for the app
CI/CD auth	Token in GitHub secret	GitHub OIDC federated credential → Azure login → Databricks azure-cli auth

Step 1: Azure SQL source with Change Tracking

► Terminal · repo root

```
az sql server create -n $SQLSRV -g $RG -l $LOC --admin-user sqladminuser --admin-password \
  '[Strong-Passw0rd!]'
# Entra admin
az sql server ad-admin create -g $RG -s $SQLSRV --display-name "[your name]" --object-id $ME
az sql server firewall-rule create -g $RG -s $SQLSRV -n allow-azure --start-ip-address \
  0.0.0.0 --end-ip-address 0.0.0.0
az sql db create -g $RG -s $SQLSRV -n shopdb --edition GeneralPurpose --compute-model \
  Serverless \
  --family Gen5 --capacity 1 --auto-pause-delay 60 --use-free-limit \
  --free-limit-exhaustion-behavior AutoPause
```

Table (source_db/schema_sqlserver.sql)	Columns (SQL Server types)
dbo.customers	customer_id bigint IDENTITY primary key · email nvarchar(320) unique · full_name nvarchar(200) · city nvarchar(100) · country nvarchar(100) · segment nvarchar(20) check in (retail, business, vip) · created_at datetime2 default sysutcdatetime() · updated_at datetime2
dbo.orders	order_id bigint IDENTITY primary key · customer_id bigint foreign key · status nvarchar(20) check · amount decimal(10,2) · currency char(3) · created_at datetime2 · updated_at datetime2
Database settings	Change Tracking on (retention 3 days, auto cleanup); snapshot isolation allowed (consistent extraction)
Per table	Change Tracking enabled with track columns updated off

► sqlcmd / Azure SQL query editor (after running schema_sqlserver.sql)

```
ALTER DATABASE shopdb SET ALLOW_SNAPSHOT_ISOLATION ON;
ALTER DATABASE shopdb SET CHANGE_TRACKING = ON (CHANGE_RETENTION = 3 DAYS, AUTO_CLEANUP = ON);
ALTER TABLE dbo.customers ENABLE CHANGE_TRACKING WITH (TRACK_COLUMNS_UPDATED = OFF);
ALTER TABLE dbo.orders ENABLE CHANGE_TRACKING WITH (TRACK_COLUMNS_UPDATED = OFF);
SELECT CHANGE_TRACKING_CURRENT_VERSION() AS current_version;
```

► Terminal · repo root

```
# ODBC driver for Python (pyodbc)
brew tap microsoft/mssql-release https://github.com/Microsoft/homebrew-mssql-release && brew \
  install msodbcsql18
# Ubuntu: follow Microsoft's "Install the ODBC driver for SQL Server on Linux" page, package
# msodbcsql18
pip install pyodbc azure-identity
# -G = Entra auth
sqlcmd -S $SQLSRV.database.windows.net -d shopdb -G -i source_db/schema_sqlserver.sql
```

Simulators: keep `seed_source.py` and `generate_changes.py`, add a `--engine sqlserver` option: connect with pyodbc (driver ODBC Driver 18, Encrypt=yes, Authentication=ActiveDirectoryDefault so your az login is used), use SQL Server syntax (OUTPUT inserted.id instead of RETURNING), and make deletes **real DELETE statements** (orders first, then the customer).

Step 2: the Function App extractor

```
functions/cdc_extractor/
├─ function_app.py          # one timer function: extract_changes (every 5 minutes)
├─ extractor_core.py        # pure logic, shared with local runs and unit tests
├─ host.json                # function timeout, logging levels
├─ requirements.txt         # azure-functions, azure-identity, azure-storage-file-datalake, pyodbc
└─ local.settings.json     # git-ignored local settings
```

functions/cdc_extractor (logic of extract_changes)

Trigger	Timer, schedule every 5 minutes; singleton behaviour (timer triggers run once per app) so runs never overlap.
Settings	SQL_SERVER, SQL_DATABASE=shopdb, STORAGE_ACCOUNT, CONTAINER=landing, LANDING_PREFIX=cdc/[env], TABLES=customers,orders, ENV
Identity	System-assigned managed identity: SQL user FROM EXTERNAL PROVIDER with db_datareader and VIEW CHANGE TRACKING; Storage Blob Data Contributor on the storage account. No passwords anywhere.
Logic	1. Read the state blob landing/cdc/[env]/_state/state.json: last_sync_version per table (missing → full load). 2. Open a SQL connection with an Entra access token for the managed identity; start a SNAPSHOT isolation transaction. 3. current = CHANGE_TRACKING_CURRENT_VERSION(). For each table, min_valid = CHANGE_TRACKING_MIN_VALID_VERSION of that table. If last_sync_version is missing or lower than min_valid → full load: select all rows with op U and seq = current. 4. Otherwise select from CHANGETABLE(CHANGES dbo.[table], last_sync_version) left joined to the table on the primary key: op = D when SYS_CHANGE_OPERATION is D, else U; seq = SYS_CHANGE_VERSION; the row's current column values (null for deletes). 5. Commit the transaction. Serialize rows as JSON Lines with extracted_at (UTC), datetimes in ISO-8601, decimals as strings. 6. Upload each non-empty file to landing/cdc/[env]/[table]/[table]_[timestamp].jsonl (no overwrite). 7. Write source_stats (active row counts, order amount sum, version = current) to landing/cdc/[env]/_stats/. 8. Only after all uploads succeed, write the new state.json with last_sync_version = current for each table. 9. Log rows per table and the new version; any exception is logged and re-raised so the run shows as failed in Application Insights.
Note for README	Change Tracking returns the net change per row between two versions, so SCD2 history has one version per extraction interval, not every intermediate update. Explain this trade-off versus CDC or Lakeflow Connect.
Done when	Deleting a customer in SQL produces an op D record and the row disappears from silver.customers.

► Terminal · repo root

```
az functionapp create -g $RG -n $FUNC --storage-account $ST --flexconsumption-location $LOC \
  --runtime python --runtime-version 3.11
az functionapp identity assign -g $RG -n $FUNC
export FUNC_PID=$(az functionapp identity show -g $RG -n $FUNC --query principalId -o tsv)
az role assignment create --assignee-object-id $FUNC_PID --assignee-principal-type \
  ServicePrincipal \
  --role "Storage Blob Data Contributor" --scope $ST_ID
az functionapp config appsettings set -g $RG -n $FUNC --settings \
  SQL_SERVER=$SQLSRV.database.windows.net SQL_DATABASE=shopdb STORAGE_ACCOUNT=$ST \
  CONTAINER=landing LANDING_PREFIX=cdc/dev TABLES=customers,orders ENV=dev
```

► sqlcmd -G (as the Entra admin) / Azure SQL query editor

```
CREATE USER [func-dbx-portfolio-4821] FROM EXTERNAL PROVIDER;      -- the Function App name
ALTER ROLE db_datareader ADD MEMBER [func-dbx-portfolio-4821];
GRANT VIEW CHANGE TRACKING ON SCHEMA::dbo TO [func-dbx-portfolio-4821];
```

► Terminal · repo root

```
cd functions/cdc_extractor
# local run (uses local.settings.json and your az login)
func start
func azure functionapp publish $FUNC          # deploy
az functionapp function list -g $RG -n $FUNC -o table
az storage fs file list -f landing --path cdc/dev --account-name $ST --auth-mode login -o table
```

Step 3: pipeline on ADLS

► SQL editor · Azure Databricks

```
CREATE EXTERNAL VOLUME IF NOT EXISTS portfolio.cdc_landing.changes
  LOCATION 'abfss://landing@stdbxportfolio[SUFFIX].dfs.core.windows.net/cdc';
```

The pipeline reads /Volumes/portfolio/cdc_landing/changes/dev/... exactly as before. In silver, the delete rule stays op = D, now driven by real deletes. Point freshness_check and reconcile_with_source at the same Volume.

Step 4: serve customer summaries from Cosmos DB

► Terminal · repo root

```
az cosmosdb create -n $COSMOS -g $RG --enable-free-tier true --default-consistency-level \
  Session \
  --locations regionName=$LOC failoverPriority=0 isZoneRedundant=false
az cosmosdb sql database create -a $COSMOS -g $RG -n appdb
az cosmosdb sql container create -a $COSMOS -g $RG -d appdb -n customer_360 \
  --partition-key-path "/customer_id" --throughput 400
az keyvault secret set --vault-name $KV -n cosmos-endpoint --value \
  https://$COSMOS.documents.azure.com:443/
az keyvault secret set --vault-name $KV -n cosmos-key \
  --value "$(az cosmosdb keys list -n $COSMOS -g $RG --type keys --query primaryMasterKey -o \
  tsv)"
```

notebooks/publish_to_cosmos

Runs on	A job cluster (single node D4ds_v5) with the Maven library com.azure.cosmos.spark:azure-cosmos-spark_3-5_2-12 matching its Spark version.
----------------	--

Logic	1. Read endpoint and key from the kv-portfolio scope. 2. Read gold.customer_order_summary; build documents: id = customer_id as string, customer_id as string (the partition key), segment, city, order_count, delivered_count, total_amount (as double), last_order_at (ISO string), is_vip, updated_at. 3. Only publish rows changed since the last publish: compare with a stored hash of each row in cdc_ops.cosmos_publish_state (customer_id, row_hash); new or changed hashes are sent. 4. Write with format cosmos.oltp: upsert (item overwrite), bulk enabled, a throughput-control limit of about 50% so the app isn't starved. 5. Customers deleted in silver: delete their documents (item delete strategy) using the ids missing from the summary. 6. Merge the new hashes into the state table; log counts upserted and deleted.
Done when	The Data Explorer in the portal shows documents whose counts match gold.

Step 5: CI/CD with OIDC (no stored Azure secrets)

► Terminal · repo root

```
az ad app create --display-name gh-dbx-postgres-cdc
export APP_ID=$(az ad app list --display-name gh-dbx-postgres-cdc --query [0].appId -o tsv)
az ad sp create --id $APP_ID
az ad app federated-credential create --id $APP_ID --parameters '{
  "name": "gh-main", "issuer": "https://token.actions.githubusercontent.com",
  "subject": "repo:[github-user]/databricks-postgres-cdc-cicd:ref:refs/heads/main",
  "audiences": ["api://AzureADTokenExchange"] }'
az role assignment create --assignee $APP_ID --role Contributor --scope $(az group show -n \
  $RG --query id -o tsv)

gh secret set AZURE_CLIENT_ID      --body $APP_ID
gh secret set AZURE_TENANT_ID      --body $(az account show --query tenantId -o tsv)
gh secret set AZURE_SUBSCRIPTION_ID --body $(az account show --query id -o tsv)
gh secret set DATABRICKS_HOST      --body $DBW_URL
```

- 1 Add the service principal to the Azure Databricks workspace (Settings → Identity and access → Service principals → add by application ID) and give it workspace access plus the Unity Catalog rights it needs to deploy.
- 2 In deploy.yml: grant the job id-token write permission; step azure/login with the three AZURE_* secrets; set DATABRICKS_AUTH_TYPE=azure-cli and DATABRICKS_HOST; then run the bundle deploy as before.
- 3 Add a job that deploys the Function App on changes under functions/ (func azure functionapp publish, or the Azure/functions-action).
- 4 Add a second federated credential with subject repo:[user]/[repo]:pull_request for dev deployments from PRs.

Screenshots for the README (Azure section)

- ☐ Change Tracking query showing a D operation
- ☐ Function App invocations in Application Insights
- ☐ Change files in ADLS
- ☐ Customer history and a real delete in silver
- ☐ Cosmos DB Data Explorer with customer_360
- ☐ GitHub Actions run using Azure OIDC login

Project 3 acceptance checklist

- ☐ Source DB with trigger and soft deletes
- ☐ Seed and change simulators
- ☐ Watermark extractor, at-least-once
- ☐ SCD1 and SCD2 for both entities
- ☐ Gold views
- ☐ Tests $\geq$ 70% coverage
- ☐ Bundle dev + prod
- ☐ CI, dev deploy on PR, gated prod deploy
- ☐ Freshness and reconciliation alerts
- ☐ Demo screenshots, README, blog

PART A

Standards and Shared Set-up

Do this part once. All projects depend on it.

A1. Naming standards

Object	Rule	Examples
GitHub repos	lowercase, hyphen-separated	databricks-ecommerce-lakehouse
Catalog	one for all portfolio work	portfolio
Schemas	[project]_[layer]; prefixes ecom, perf, cdc	ecom_bronze, perf_gold, cdc_silver
Tables	snake_case; gold uses dim_ / fact_ / agg_	orders, dim_customer, agg_daily_sales
Quarantine tables	[table]_quarantine, same schema	ecom_silver.orders_quarantine
Technical columns	leading underscore	_ingested_at, _source_file, _batch_id
Time columns	_at for timestamps, _date for dates	purchased_at, order_date
Pipelines / jobs	[project]_[purpose]_pipeline / _job	ecom_lakehouse_pipeline, ecom_daily_job
Branches	main + feature/[short-name]	feature/silver-layer
Commits	type: summary (feat, fix, docs, test, refactor, chore, perf)	feat: add SCD2 customer dimension

A2. Accounts to create

Account	Used for	Notes
GitHub	Repos, profile, CI/CD	Turn on two-factor authentication
Databricks Free Edition	Pipelines, jobs, dashboards	Sign up with your professional email
Kaggle	Olist dataset (Project 1)	Account → Settings → API → “Create New Token” downloads kaggle.json
Neon (or Supabase)	Hosted PostgreSQL (Project 3)	Alternative: Docker PostgreSQL locally
draw.io / Excalidraw	Diagrams	Browser, no install
Medium / Hashnode / dev.to	Blog posts	Pick one
Microsoft Azure (free account)	Azure tracks and Project 4	Set a budget alert first (A9)

A3. Install local tools

Step 1: Git, Python 3.11 and Java 17

► Terminal · anywhere

```
# macOS (Homebrew)
brew install git python@3.11 openjdk@17
sudo ln -sfn "$(brew --prefix)/opt/openjdk@17/libexec/openjdk.jdk" \
/Library/Java/JavaVirtualMachines/openjdk-17.jdk

# Ubuntu / WSL
sudo apt update && sudo apt install -y git python3.11 python3.11-venv openjdk-17-jdk

# Windows (PowerShell)
winget install Git.Git Python.Python.3.11 EclipseAdoptium.Temurin.17.JDK
```

► Terminal · verify

```
git --version          # expect 2.40+
python3.11 --version   # expect 3.11.x   (Windows: py -3.11 --version)
java -version          # expect 17.x
```

If PySpark later says it cannot find Java, set JAVA_HOME in your shell profile (~/.zshrc or ~/.bashrc):

► Terminal · anywhere

```
# macOS
echo 'export JAVA_HOME=$(/usr/libexec/java_home -v 17)' >> ~/.zshrc && source ~/.zshrc
# Ubuntu / WSL
echo 'export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64' >> ~/.bashrc && source ~/.bashrc
```

Step 2: Git identity and SSH key for GitHub

► Terminal · anywhere

```
git config --global user.name "[Your Name]"
git config --global user.email "[you@yourbrand.com]"
git config --global init.defaultBranch main

ssh-keygen -t ed25519 -C "[you@yourbrand.com]" # press Enter to accept defaults
cat ~/.ssh/id_ed25519.pub                     # copy the output
# Browser: GitHub → Settings → SSH and GPG keys → New SSH key → paste
# expect: "Hi [username]! You've successfully authenticated"
ssh -T git@github.com
```

Step 3: Databricks CLI

► Terminal · anywhere

```
# macOS / Linux / WSL
curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
# or macOS: brew tap databricks/tap && brew install databricks
# Windows: winget install Databricks.DatabricksCLI

databricks -v # expect v0.2xx or newer
```

Step 4: VS Code and extensions

- Install VS Code, then the extensions: **Python**, **Databricks**, **YAML**, **Ruff**, **GitHub Actions**, **Rainbow CSV**.
- Open a repo folder with code . from its root.

Step 5: optional tools

► Terminal · anywhere

```
brew install --cask docker # Docker Desktop (Project 3 local Postgres alternative)
# psql client (Ubuntu: sudo apt install postgresql-client)
brew install libpq && brew link --force libpq
brew install gh # GitHub CLI (optional, for secrets and PRs from terminal)
```

Step 6: folder for all projects

► Terminal · anywhere

```
mkdir -p ~/portfolio && cd ~/portfolio
```

A4. Connect the CLI to Databricks

- 1 Sign in to Databricks Free Edition in the browser. Copy the workspace URL from the address bar (everything up to and including .com or .net), for example `https://dbc-1234abcd-5678.cloud.databricks.com`.
- 2 Log in from the terminal with a named profile `portfolio`. A browser window opens; approve it.

► Terminal · anywhere

```
databricks auth login --host https://[your-workspace-url] --profile portfolio
databricks auth profiles # 'portfolio' should show Valid: YES
databricks current-user me --profile portfolio
export DATABRICKS_CONFIG_PROFILE=portfolio # add to ~/.zshrc so every command uses it
```

Fallback if browser login is not available: create a personal access token in **Settings → Developer → Access tokens → Generate new token** (lifetime 90 days) and run:

► Terminal · anywhere

```
databricks configure --token --profile portfolio # paste host, then token
```

A5. Create the catalog, schemas and Volumes

Run either the SQL version (in the Databricks SQL editor) or the CLI version (terminal). Both produce the same result.

► SQL editor

```
CREATE CATALOG IF NOT EXISTS portfolio;

CREATE SCHEMA IF NOT EXISTS portfolio.ecom_landing;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_silver;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_ops;

CREATE SCHEMA IF NOT EXISTS portfolio.perf_raw;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_bench;

CREATE SCHEMA IF NOT EXISTS portfolio.cdc_landing;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_silver;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_ops;

CREATE VOLUME IF NOT EXISTS portfolio.ecom_landing.raw;
-- raw files for exploration only
CREATE VOLUME IF NOT EXISTS portfolio.ecom_landing.profiling;
CREATE VOLUME IF NOT EXISTS portfolio.perf_raw.tlc;
CREATE VOLUME IF NOT EXISTS portfolio.cdc_landing.changes;

SHOW SCHEMAS IN portfolio;
SHOW VOLUMES IN portfolio.ecom_landing;
```

► Terminal • alternative to SQL

```
databricks catalogs create portfolio
for s in ecom_landing ecom_bronze ecom_silver ecom_gold ecom_ops \
    perf_raw perf_bronze perf_gold perf_bench \
    cdc_landing cdc_bronze cdc_silver cdc_gold cdc_ops; do
  databricks schemas create "$s" portfolio
done
databricks volumes create portfolio ecom_landing raw MANAGED
databricks volumes create portfolio ecom_landing profiling MANAGED
databricks volumes create portfolio perf_raw tlc MANAGED
databricks volumes create portfolio cdc_landing changes MANAGED
databricks schemas list portfolio
```

WATCH OUT

If Free Edition refuses `CREATE CATALOG`, run `SHOW CATALOGS;`, use the workspace's default catalog instead of `portfolio` everywhere in this manual, and keep all schema names the same. Also note: Free Edition runs on serverless compute with daily quotas and may block outbound internet from notebooks, so every internet-dependent step here runs on your laptop.

A6. Link GitHub to Databricks

- 1 Browser: Databricks → your profile icon → **Settings** → **Linked accounts** → **Git integration** → provider GitHub → link (OAuth app, or a GitHub fine-grained token with repo contents read/write).
- 2 Each time you create a repo: Workspace → your user folder → **Create** → **Git folder** → paste the repo HTTPS URL. Pull changes from the Git folder's branch menu after every local push.

A7. Standard files in every repo

File	Exact contents to put in it
<code>.gitignore</code>	Lines: <code>.env/ __pycache__/* .pyc .env data/ .databricks/* .log .pytest_cache/ .coverage htmlcov/ .DS_Store</code>
<code>.env.example</code>	One setting per line with an empty value: <code>DATABRICKS_CONFIG_PROFILE=portfolio</code> , plus project-specific keys listed in each project
<code>requirements-dev.txt</code>	<code>pyspark==3.5.* · pytest · pytest-cov · chispa · ruff · databricks-sdk · python-dotenv · pandas · pyarrow · requests</code> , plus project extras
<code>pyproject.toml</code>	[project] name and <code>requires-python = ">=3.11"</code> ; [tool.ruff] <code>line-length = 110</code> , <code>target-version py311</code> ; [tool.pytest.ini_options] <code>testpaths = ["tests"]</code> , <code>pythonpath = ["src"]</code>
<code>LICENSE</code>	MIT (choose it when creating the repo on GitHub)
<code>README.md</code>	Structure in Part F

Standard commands you'll use in every repo

► Terminal · repo root

```
# one-time per repo
python3.11 -m venv .venv
source .venv/bin/activate                                # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements-dev.txt                       # then fill in values
cp .env.example .env

# every working session
source .venv/bin/activate
git pull
ruff check . && ruff format --check .                    # lint
pytest -q                                                # tests

# feature workflow
git checkout -b feature/[short-name]
git add -A && git commit -m "feat: [summary]"
git push -u origin feature/[short-name]
# GitHub: open a pull request, merge it, then:
git checkout main && git pull && git branch -d feature/[short-name]
```

A8. The testable-code pattern

All transformation logic lives in plain functions in `src/[package]/transformations/`: each takes a `DataFrame` (plus parameters) and returns a `DataFrame`, with no reading or writing of tables inside. Pipeline files only read, call functions, and declare tables. That's what lets `pytest` run everything locally with a small in-memory `SparkSession`, and it keeps pipeline files short enough for a client to read.

A9. Azure set-up (for the Azure tracks and Project 4)

Every project in this manual has an **Azure track** that swaps free tools for the Azure services clients actually use: **ADLS Gen2**, **Azure SQL Database**, **Cosmos DB**, **Function Apps**, **Virtual Machines**, plus Key Vault. Build each project on Databricks Free Edition first, then do its Azure track in one focused session and delete the resources afterwards. Do this section once, just before your first Azure track.

Azure resource names (used everywhere below)

Resource	Name	Notes
Resource group	rg-dbx-portfolio	Everything lives here; delete it to remove everything
Region	westeurope	Or your nearest region; keep everything in one region
Storage (ADLS Gen2)	stdbxportfolio\$SUFFIX	Containers landing and lake
Access Connector	ac-dbx-portfolio	Managed identity Databricks uses to reach ADLS
Azure Databricks	dbw-portfolio	Trial pricing tier; CLI profile azure
Key Vault	kv-dbx-portfolio-\$SUFFIX	Databricks secret scope kv-portfolio
Azure SQL	sql-dbx-portfolio-\$SUFFIX	Databases shopdb (Project 3) and servingdb (Projects 1 and 4)
Cosmos DB	cosmos-dbx-portfolio-\$SUFFIX	Database appdb; containers customer_profiles, customer_360
Function App	func-dbx-portfolio-\$SUFFIX	Flex Consumption, Python 3.11, system-assigned identity
Virtual Machine	vm-legacy-portfolio	Only in Project 4 (legacy SQL Server + self-hosted runtime)

Resource	Name	Notes
Log Analytics	log-dbx-portfolio	Diagnostic logs (Project 4)

Step 1: variables, login and budget

► Terminal · ~/portfolio/azure.env

```
# put these in ~/portfolio/azure.env and run: source ~/portfolio/azure.env
(SUFFIX = 3-5 random digits)
export SUFFIX=4821
export RG=rg-dbx-portfolio LOC=westeurope
export ST=stdbxportfolio$SUFFIX KV=kv-dbx-portfolio-$SUFFIX
export SQLSRV=sql-dbx-portfolio-$SUFFIX COSMOS=cosmos-dbx-portfolio-$SUFFIX \
  FUNC=func-dbx-portfolio-$SUFFIX
# your Entra object id (after az login)
export ME=$(az ad signed-in-user show --query id -o tsv)
```

► Terminal · repo root

```
az login
az account set --subscription "[subscription name or id]"
source ~/portfolio/azure.env
az group create -n $RG -l $LOC --tags owner=[you] env=lab project=portfolio
az extension add -n databricks
```

Browser: **Cost Management** → **Budgets** → **Add**: scope rg-dbx-portfolio, \$20 per month, alerts at 50/80/100% to your email. Do it before creating anything else.

Step 2: ADLS Gen2, Access Connector and Key Vault

► Terminal · repo root

```
az storage account create -n $ST -g $RG -l $LOC --sku Standard_LRS --kind StorageV2 --hns true \
  --min-tls-version TLS1_2 --allow-blob-public-access false
export ST_ID=$(az storage account show -n $ST -g $RG --query id -o tsv)
az role assignment create --assignee-object-id $ME --assignee-principal-type User \
  --role "Storage Blob Data Contributor" --scope $ST_ID
az storage fs create -n landing --account-name $ST --auth-mode login
az storage fs create -n lake --account-name $ST --auth-mode login

az databricks access-connector create -n ac-dbx-portfolio -g $RG -l $LOC --identity-type \
  SystemAssigned
export AC_ID=$(az databricks access-connector show -n ac-dbx-portfolio -g $RG --query id -o tsv)
export AC_PID=$(az databricks access-connector show -n ac-dbx-portfolio -g $RG --query \
  identity.principalId -o tsv)
az role assignment create --assignee-object-id $AC_PID --assignee-principal-type \
  ServicePrincipal \
  --role "Storage Blob Data Contributor" --scope $ST_ID

az keyvault create -n $KV -g $RG -l $LOC --enable-rbac-authorization true
export KV_ID=$(az keyvault show -n $KV --query id -o tsv)
az role assignment create --assignee-object-id $ME --assignee-principal-type User \
  --role "Key Vault Secrets Officer" --scope $KV_ID
# AzureDatabricks first-party application must read secrets for the Key Vault-backed scope
az role assignment create --assignee 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d \
  --role "Key Vault Secrets User" --scope $KV_ID
# note the DNS name for the scope
az keyvault show -n $KV --query properties.vaultUri -o tsv
```

Step 3: Azure Databricks workspace and Unity Catalog

► Terminal · repo root

```
az databricks workspace create -n dbw-portfolio -g $RG -l $LOC --sku trial
export DBW_URL=https://$(az databricks workspace show -n dbw-portfolio -g $RG --query \
  workspaceUrl -o tsv)
databricks auth login --host $DBW_URL --profile azure
databricks current-user me --profile azure
# vCPU quota for clusters
az vm list-usage -l $LOC -o table | grep -i -E "total regional|DDSV5"
```

- 1 Browser, Azure Databricks: **Catalog → External data → Credentials → Create credential** → Azure Managed Identity → Access Connector ID = the value of \$AC_ID → name cred_adls_portfolio.
- 2 Create the Key Vault-backed secret scope: open [DBW_URL]#secrets/createScope, scope name kv-portfolio, DNS name = vault URI, resource ID = \$KV_ID.
- 3 Compute: create a cluster policy or personal compute with **single node, Standard_D4ds_v5, auto-termination 20 minutes**. Free-trial subscriptions have small vCPU quotas.
- 4 Run the SQL below, then the schema and Volume SQL from A5 (skip its CREATE CATALOG line).

► SQL editor · Azure Databricks

```
CREATE EXTERNAL LOCATION IF NOT EXISTS loc_landing
  URL 'abfss://landing@stdbxportfolio[SUFFIX].dfs.core.windows.net/' WITH (STORAGE CREDENTIAL
    cred_adls_portfolio);
CREATE EXTERNAL LOCATION IF NOT EXISTS loc_lake
  URL 'abfss://lake@stdbxportfolio[SUFFIX].dfs.core.windows.net/' WITH (STORAGE CREDENTIAL
    cred_adls_portfolio);
CREATE CATALOG IF NOT EXISTS portfolio
  MANAGED LOCATION 'abfss://lake@stdbxportfolio[SUFFIX].dfs.core.windows.net/portfolio';
SHOW EXTERNAL LOCATIONS;
```

WATCH OUT

Cost discipline: terminate clusters when you stop, keep auto-termination on, deallocate VMs, and when the Azure track is finished run `az group delete -n $RG --yes --no-wait`. Keep screenshots; they are the portfolio evidence, not the running resources. For the next Azure track, repeat Steps 1–3 (about 20 minutes).

Python packages for Azure tracks (add to requirements-dev.txt when needed)

azure-identity (DefaultAzureCredential uses your az login locally and managed identity in Azure) ·
 azure-storage-file-datalake · azure-keyvault-secrets · azure-cosmos · pyodbc (plus the Microsoft ODBC
 Driver 18 for SQL Server) · azure-functions (inside the Function App project only).

A10. Uploading data into Databricks (Volumes and tables)

In Unity Catalog, **files** live in **Volumes** (governed folders such as /Volumes/portfolio/ecom_landing/raw) and **tables** are Delta tables in a schema. The pipelines in this manual read files from Volumes with Auto Loader, so the main task is getting files into the right Volume folder. Pick the method that fits the job:

#	Method	Best for	Used in
1	Catalog Explorer: Upload to this volume (browser)	A few files, quick exploration	Project 1 profiling files
2	Add data → Create or modify table (browser)	One CSV/JSON/Parquet file straight into a table	Quick sandbox tables
3	Databricks CLI databricks fs cp	Folders and larger files from your laptop	All projects
4	Python SDK scripts (upload_batch.py, upload_tlc.py)	Repeatable, logged uploads	Projects 1 and 2
5	SQL : read_files, COPY INTO	Turning Volume files into tables	Exploration, checks
6	Notebook : %sh curl / dbutils.fs.cp	Downloading from the internet inside Databricks	Project 2 (if internet is allowed)

#	Method	Best for	Used in
7	Azure: azcopy to ADLS Gen2 + external Volume	Client-style landing zones	Azure tracks, Project 4

First: create a sandbox schema for exploration tables

► SQL editor

```
CREATE SCHEMA IF NOT EXISTS portfolio.sandbox COMMENT 'Throw-away tables for exploration';
```

Method 1: upload files to a Volume in the browser

- 1 Left sidebar → **Catalog** → portfolio → ecom_landing → **Volumes** → profiling.
- 2 Optional: **Create directory** (top right) to make a sub-folder, for example olist.
- 3 Click **Upload to this volume**, drag the 9 CSV files from data/raw into the dialog, and click **Upload**.
- 4 Check the files are listed in the Volume page, or run the LIST command below.
- 5 Copy a file's path with the **⋮** → **Copy path** menu when you need it in a notebook.

► SQL editor · verify

```
LIST '/Volumes/portfolio/ecom_landing/profiling/';
```

Method 2: create a table directly from a file (browser)

- 1 Left sidebar → **New (+)** → **Add or upload data** → **Create or modify table**.
- 2 Drop one file (for example olist_orders_dataset.csv).
- 3 Choose catalog portfolio, schema sandbox, table name orders_raw.
- 4 Check the preview: first row as header on; for the reviews file open **Advanced attributes** and enable **multi-line** so comments with line breaks stay in one row.
- 5 Adjust column types if needed, then **Create table**. The result is a managed Delta table you can query at once.

TIP

Use Method 2 only for exploration. Pipelines must read files from Volumes, so production-style flows use Methods 3, 4 or 7.

Method 3: Databricks CLI (from your laptop)

► Terminal · repo root

```
# one file
databricks fs cp data/raw/olist_orders_dataset.csv \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv
# a whole folder (recursive)
databricks fs cp -r data/raw dbfs:/Volumes/portfolio/ecom_landing/profiling/
# one batch into the landing folders the pipeline reads
databricks fs cp -r data/batches/2016_09/orders dbfs:/Volumes/portfolio/ecom_landing/raw/orders/
# replace an existing file (never do this in raw landing folders)
databricks fs cp --overwrite data/raw/olist_orders_dataset.csv \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv
# inspect and clean up
databricks fs mkdir dbfs:/Volumes/portfolio/ecom_landing/profiling/olist
databricks fs ls dbfs:/Volumes/portfolio/ecom_landing/profiling/
databricks fs cat \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/product_category_name_translation.csv
databricks fs rm -r dbfs:/Volumes/portfolio/ecom_landing/profiling/olist
```

Method 4: the upload scripts (repeatable and logged)

► Terminal · repo root

```
python scripts/upload_batch.py --batch 0000_initial      # Project 1: reference data
python scripts/upload_batch.py --next                  # Project 1: next monthly batch
python scripts/upload_batch.py --all --dry-run          # show what would be uploaded
# Project 2: TLC Parquet + zone lookup
python scripts/upload_tlc.py
```

Method 5: SQL on files in Volumes

► SQL editor

```
-- peek at files without creating anything
SELECT * FROM read_files('/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv',
                        format => 'csv', header => true) LIMIT 10;

-- create a table from a file (types inferred)
CREATE OR REPLACE TABLE portfolio.sandbox.orders_raw AS
SELECT * FROM read_files('/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv',
                        format => 'csv', header => true, inferSchema => true);

-- multi-line CSV (review comments contain line breaks and quotes)
CREATE OR REPLACE TABLE portfolio.sandbox.reviews_raw AS
SELECT * FROM
  read_files('/Volumes/portfolio/ecom_landing/profiling/olist_order_reviews_dataset.csv',
            format => 'csv', header => true, multiLine => true, escape => '');

-- Parquet folder (Project 2)
SELECT COUNT(*) FROM read_files('/Volumes/portfolio/perf_raw/tlc/yellow/', format => 'parquet');

-- incremental, idempotent load: re-running COPY INTO only loads files it has not loaded before
CREATE TABLE IF NOT EXISTS portfolio.sandbox.orders_copy;
COPY INTO portfolio.sandbox.orders_copy
FROM '/Volumes/portfolio/ecom_landing/raw/orders/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
SELECT COUNT(*) FROM portfolio.sandbox.orders_copy;
```

Method 6: download inside a notebook (only if outbound internet is allowed)

► Notebook cell (%sh)

```
%sh
mkdir -p /Volumes/portfolio/perf_raw/tlc/zones
curl -L -o /Volumes/portfolio/perf_raw/tlc/zones/taxi_zone_lookup.csv \
  https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv
ls -lh /Volumes/portfolio/perf_raw/tlc/zones/
```

► Notebook cell (Python)

```
dbutils.fs.ls("/Volumes/portfolio/perf_raw/tlc/zones/")
dbutils.fs.cp("/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv",
              "/Volumes/portfolio/ecom_landing/profiling/copy/orders.csv") # copy a file
```

Method 7: Azure landing zone (Azure tracks)

Upload with azcopy or the Data Lake SDK to the ADLS container, then expose it through an external Volume (commands in A9, B13 and C8). Databricks then reads it exactly like any other Volume path.

WATCH OUT

Files in the **raw landing** Volumes are immutable: Auto Loader tracks what it has processed and ignores a re-uploaded file with the same name. Upload new data as new files, and keep exploration files in the **profiling** Volume or the sandbox schema. UI uploads are meant for small files; use the CLI or scripts for large ones, and check the current upload limits in the Databricks documentation.

Part A checklist

- ☐ Can upload files to a Volume (A10) and create a table from them
- ☐ Tools installed and verified
- ☐ Git identity and SSH key working
- ☐ CLI profile 'portfolio' valid
- ☐ Catalog, 14 schemas and 3 Volumes exist
- ☐ GitHub linked to Databricks

PART F

Finishing, Publishing and Troubleshooting

F1. README structure

Section	Content
Title + badges	Name; CI badge; tech badges
Business problem	Client story, 2-3 sentences
Results	3-5 bullets with measured numbers
Architecture	Diagram + walkthrough
Data model	Table catalogue and diagram
Design decisions	Choices and alternatives
Screenshots	Graph, lineage, dashboard, CI, alerts
How to run	Every command from this manual, in order
Testing	How to run tests; coverage
Limitations and next steps	Honest list

F2. Publish each project

► Terminal · repo root

```
gh repo edit --description "Databricks lakehouse: Auto Loader, Lakeflow pipelines, SCD2, \
  Unity Catalog, AI/BI" \
  --add-topic databricks --add-topic pyspark --add-topic delta-lake --add-topic \
  data-engineering
git tag -a v1.0 -m "Portfolio release" && git push origin v1.0
gh release create v1.0 --title "v1.0" --notes "First complete version"
```

- 1 GitHub profile → Customize your pins → select the repo.
- 2 Publish the blog post; post on LinkedIn (hook with the result, 4-5 points, diagram; repo link in first comment).
- 3 Add to the Upwork portfolio and the LinkedIn Featured section.

F3. Troubleshooting

Problem	Cause	Fix / command
CLI: "cannot configure default credentials"	No profile selected	export DATABRICKS_CONFIG_PROFILE=portfolio, or add --profile portfolio
Pipeline: ModuleNotFoundError ecom	src not on the path	Set the pipeline root folder to the repo's src, or add the path in the pipeline source settings
Review rows broken across lines	Multi-line text	multiLine plus quote and escape options for order_reviews
Silver drops nearly everything	Rule checked before cleaning, or case mismatch	Apply rules after type_ functions; unit-test the rules
Re-uploaded file ignored	Auto Loader skips processed files	New files need new names; landing is immutable
Need to rebuild everything	Logic changed for existing data	databricks bundle run -t dev [pipeline] --full-refresh-all

Problem	Cause	Fix / command
SCD2 versions out of order	Wrong or non-unique sequence	Sequence by change_ts / seq; break ties with the key
pytest: Java gateway exited	Java missing or JAVA_HOME wrong	java -version; set JAVA_HOME (A3)
CI: bundle validate auth error	Secret names differ	gh secret list; names must match the workflow exactly
psql: SSL required	Neon needs SSL	Keep sslmode=require in PG_CONNECTION
Free Edition quota reached	Heavy compute in one day	Fewer TLC months; run experiments locally; continue next day
Notebook can't reach internet/Postgres	Outbound access restricted	Run on the laptop and upload to a Volume
Azure: 403 AuthorizationPermissionMismatch on ADLS	Control-plane role instead of data role, or role not propagated yet	Assign Storage Blob Data Contributor; wait up to 10 minutes; check the external location's credential
Azure Databricks: cluster fails with QuotaExceeded	vCPU quota for the VM family is too low	az vm list-usage -l \$LOC -o table; use a smaller single-node size or request more quota in the portal (Quotas)
Azure SQL: login failed / client IP not allowed	Firewall rule missing	az sql server firewall-rule create ...; for serverless Databricks compute configure network access
Azure SQL: CREATE USER FROM EXTERNAL PROVIDER fails	Not connected as the Entra admin	sqlcmd -G as the Entra admin set with az sql server ad-admin create
Cosmos DB: 429 Too Many Requests	Writes exceed provisioned RU/s	Enable bulk and throughput control; raise RU/s temporarily
Function App: runs but writes nothing	Managed identity lacks the data role, or wrong app settings	az functionapp identity show; check role assignments and az functionapp config appsettings list
Secret scope: cannot read Key Vault	AzureDatabricks app lacks Key Vault Secrets User	Assign the role to app id 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d
Unexpected Azure bill	Clusters or VM left running	Auto-termination, az vm deallocate, delete the resource group after each track
Benchmark times vary widely	Warm-up and shared resources	3 runs, median, drop outputs before each run

PART G

Command Reference

Every command used in this manual, grouped by tool, for quick lookup.

Git

► Terminal

```
git clone git@github.com:[user]/[repo].git
git status
git pull
git checkout -b feature/[name]
git add -A
git commit -m "type: summary"
git push -u origin feature/[name]
git checkout main
git branch -d feature/[name]
git log --oneline --graph -15
git diff
git restore [file]
git tag -a v1.0 -m "Portfolio release"
git push origin v1.0
```

Python environment and quality

► Terminal

```
python3.11 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt
pip freeze > requirements-lock.txt
ruff check .
ruff check . --fix
ruff format .
pytest -q
pytest -q -k [test_name]
pytest -q --cov=src --cov-report=term-missing
```

Databricks CLI: auth, files, pipelines, jobs

► Terminal

```
databricks auth login --host https://[workspace-url] --profile portfolio
databricks auth profiles
databricks current-user me
databricks catalogs list
databricks schemas list portfolio
databricks volumes list portfolio [schema]
databricks fs ls dbfs:/Volumes/portfolio/[schema]/[volume]/
databricks fs cp [local-file] dbfs:/Volumes/portfolio/[schema]/[volume]/[file]
databricks fs cp -r [local-folder] dbfs:/Volumes/portfolio/[schema]/[volume]/[folder]
databricks fs cp --overwrite [local-file] dbfs:/Volumes/.../[file]
databricks fs rm -r dbfs:/Volumes/portfolio/[schema]/[volume]/[folder]
databricks pipelines list-pipelines
databricks pipelines start-update [pipeline_id]
databricks pipelines get [pipeline_id]
databricks pipelines stop [pipeline_id]
databricks jobs list
databricks jobs run-now [job_id]
databricks jobs list-runs --job-id [job_id] --limit 5
```

Databricks Asset Bundles

► Terminal

```
databricks bundle init # start from a template
databricks bundle generate pipeline --existing-pipeline-id [id] --key [name]
databricks bundle generate job --existing-job-id [id] --key [name]
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev [job_or_pipeline_key]
databricks bundle run -t dev [pipeline_key] --full-refresh-all # rebuild all tables
databricks bundle summary -t dev
databricks bundle destroy -t dev # removes deployed resources
```

SQL editor: inspection and maintenance

► SQL editor

```
SHOW CATALOGS;
SHOW SCHEMAS IN portfolio;
SHOW TABLES IN portfolio.[schema];
SHOW VOLUMES IN portfolio.[schema];
DESCRIBE TABLE EXTENDED portfolio.[schema].[table];
DESCRIBE DETAIL portfolio.[schema].[table];
DESCRIBE HISTORY portfolio.[schema].[table];
SELECT * FROM portfolio.[schema].[table] VERSION AS OF [n];
OPTIMIZE portfolio.[schema].[table];
ALTER TABLE portfolio.[schema].[table] CLUSTER BY ([col1], [col2]);
COMMENT ON TABLE portfolio.[schema].[table] IS '[text]';
ALTER TABLE portfolio.[schema].[table] SET TAGS ('[key]' = '[value]');
GRANT SELECT ON SCHEMA portfolio.[schema] TO `[group]`;
SHOW GRANTS ON SCHEMA portfolio.[schema];
SELECT * FROM event_log('[pipeline_id]') ORDER BY timestamp DESC LIMIT 50;
```

Data download and PostgreSQL

► Terminal

```
kaggle datasets download -d olistbr/brazilian-ecommerce -p data/raw --unzip
curl -L -o [file] \
  https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_[YYYY]-[MM].parquet
docker run --name shopdb -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=shopdb -p 5432:5432 -d \
  postgres:16
docker start shopdb
docker stop shopdb
docker logs shopdb
set -a && source .env && set +a
psql "$PG_CONNECTION" -f source_db/schema.sql
psql "$PG_CONNECTION" -c "\dt"
```

Project scripts

► Terminal

```
# Project 1
python scripts/prepare_batches.py --raw data/raw --out data/batches [--months N] [--force]
python scripts/generate_customer_updates.py --batch YYYY_MM --pct 2 --seed 42
python scripts/make_bad_rows_batch.py --from-batch 2018_08 --out-batch 9999_bad_rows
python scripts/upload_batch.py --batch [id] | --next | --all [--dry-run]

# Project 2
python scripts/download_tlc.py --year 2023 --months 1-12 --out data/tlc
python scripts/upload_tlc.py
python local/skew_experiment.py --rows 20000000 --hot-share 0.6
python local/shuffle_partitions_experiment.py --rows 10000000

# Project 3
python simulator/seed_source.py --customers 5000 --orders 20000 --seed 7 [--reset]
python simulator/generate_changes.py --duration-min 10 --rate 5 --mix 50,40,10
python extractor/extract_changes.py --env dev|prod [--full] [--dry-run]
```

Azure CLI

► Terminal

```
az login
az account set --subscription "[name or id]"
az group create -n $RG -l $LOC
az group delete -n $RG --yes --no-wait
az storage account create -n $ST -g $RG --kind StorageV2 --hns true --sku Standard_LRS
az storage fs create -n [container] --account-name $ST --auth-mode login
az storage fs file list -f [container] --path [folder] --account-name $ST --auth-mode login \
  -o table
az role assignment create --assignee-object-id [id] --assignee-principal-type \
  [User|ServicePrincipal] --role "[role]" --scope [id]
az role assignment list -g $RG -o table
az keyvault create -n $KV -g $RG --enable-rbac-authorization true
az keyvault secret set --vault-name $KV -n [name] --value [value]
az databricks workspace create -n dbw-portfolio -g $RG --sku trial
az databricks access-connector create -n ac-dbx-portfolio -g $RG --identity-type SystemAssigned
az sql server create -n $SQLSRV -g $RG --admin-user [user] --admin-password [password]
az sql server ad-admin create -g $RG -s $SQLSRV --display-name [name] --object-id $ME
az sql server firewall-rule create -g $RG -s $SQLSRV -n [rule] --start-ip-address [ip] \
  --end-ip-address [ip]
az sql db create -g $RG -s $SQLSRV -n [db] --edition GeneralPurpose --compute-model \
  Serverless --family Gen5 \
  --capacity 1 --use-free-limit --free-limit-exhaustion-behavior AutoPause
az cosmosdb create -n $COSMOS -g $RG --enable-free-tier true
az cosmosdb sql container create -a $COSMOS -g $RG -d [db] -n [container] \
  --partition-key-path "/"[key]" --throughput 400
az functionapp create -g $RG -n $FUNC --storage-account $ST --flexconsumption-location $LOC \
  --runtime python --runtime-version 3.11
az functionapp config appsettings set -g $RG -n $FUNC --settings KEY=value
az vm create -g $RG -n [vm] --image [image] --size [size] --admin-username [user]
az vm deallocate -g $RG -n [vm]
az vm list-usage -l $LOC -o table
az deployment group what-if -g $RG -f infra/main.bicep -p infra/parameters.dev.json
az deployment group create -g $RG -f infra/main.bicep -p infra/parameters.dev.json
```

Azure data tools

► Terminal

```
azcopy login
azcopy copy "[local folder]/*" "https://$ST.dfs.core.windows.net/[container]/[path]/" \
  --recursive
sqlcmd -S $SQLSRV.database.windows.net -d [db] -G -Q "[query]" # Entra auth
sqlcmd -S $SQLSRV.database.windows.net -d [db] -U [user] -P [password] -i [file.sql]
func init [name] --python
func new --name [function] --template "Timer trigger"
func start
func azure functionapp publish $FUNC
```

GitHub CLI

► Terminal

```
gh auth login
gh repo edit --description "[text]" --add-topic [topic]
gh secret set [NAME]
gh secret set [NAME] --env production
gh secret list
gh pr create --fill
gh pr checks --watch
gh pr merge --squash --delete-branch
gh run list --limit 5
gh run view [run_id] --log-failed
gh release create v1.0 --notes "[text]"
```

Remember: finished and documented beats perfect. Publish Project 1 as soon as its checklist is done, start sending proposals, and build Projects 2 and 3 while you look for clients.